



桂林电子科技大学
GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY

编译原理课程设计 任务要求

李云辉

桂林电子科技大学
计算机与信息安全学院

2026. 05

目 录

1 引言.....	1
2 PL/0 语言的词法规则和语法规则.....	2
2.1 词法规则.....	2
2.2 语法规则.....	2
3 Flex 和 Bison 的使用.....	5
3.1 任务描述.....	5
3.2 实验.....	5
3.3 测试说明.....	6
4 词法分析程序的设计与实现.....	7
4.1 任务描述.....	7
4.2 实验.....	7
4.3 测试说明.....	8
5 语法分析程序设计与实现.....	13
5.1 任务描述.....	13
5.2 实验.....	13
5.3 测试说明.....	14
6 语义分析.....	19
6.1 任务描述.....	19
6.2 常见的四元式格式.....	19
6.3 实验.....	20
6.4 测试说明.....	20
7 课程设计的组织形式要求.....	26
8 课程设计完成形式.....	26
9 验收及考核.....	26

1 引言

(1) 问题背景

编译原理课程设计与实用编译器的相比较,实用编译器比编译原理课程设计要完成的复杂的多。

(2) 目的

通过实践（设计与开发），对编译程序的设计原理及技术进一步的掌握和了解，培养学生计算思维能力及系统能力的提升。

(3) 问题分析

编译原理课程设计做什么？

设计并实现一个简单 PL/0 语言的编译器，主要包括几个模块：

- 词法分析模块
- 语法分析模块
- 语义分析和中间代码生成模块

整个编译过程，是采用一遍扫描还是多遍扫描？

(4) 要点

遵循软件的开发过程：分析、设计、编码、集成和测试

2 PL/0 语言的词法规则和语法规则

PL/0 是瑞士计算机科学家 Niklaus Wirth 于 1975 年提出的一种教学用小型高级语言，是 Pascal 的极简子集，核心用于演示编译原理，特点是小巧、规范、易实现。

2.1 词法规则

扩展巴克斯范式（Extended Backus-Naur Form, EBNF）是程序语言语法描述的一种形式，其用到的元符号及对应的含义如表 2.1 所示。

表 2.1 EBNF 元符号及含义

元符号	含义
<>	用尖括号括起来的中文文字表示语法构造成分，或称 语法单位 ，为非终结符（<句子> ::= <主语> <谓语>）； 而用尖括号括起来的英文字表示一类 词法单元 （<ID> ::= 字母 {字母 数字}）。
::=	表示左部的语法单位由右部定义，可读作“定义为”。
	表示“或”，即左部可由多个右部定义。 <句子> ::= <主语> <谓语> <主语> <谓语> <宾语>
{ }	用花括号括起来的成分可以重复 0 次到任意多次。 <ID> ::= 字母 {字母 数字}
[]	用方括号括起来的成分为任选项，即出现一次或不出现。 <简单句> ::= <主语> [<状语>] <谓语>
()	用圆括号括起来的成分优先。 <表达式> ::= (<表达式> + <项>) <项>

2.2 语法规则

G[<程序>]:

表 2.2 PL/0 语法单位及描述（红色的为例子）

PL/0 语法单位	EBNF 描述
<程序>	::=<分程序>.
<分程序>	::=[<常量说明部分>][<变量说明部分>][<过程说明部分>]<语句>
<常量说明部分>	::= const <常量定义>{,<常量定义>;} 一个常量: const num = 100; 多个常量: const a = 10, b = 20, c = 30;
<常量定义>	::=<标识符>=<无符号整数> num = 100
<变量说明部分>	::= var <标识符>{,<标识符>;} 一个变量: var x; 多个变量: var a, b, c, total;
<过程说明部分>	::=<过程首部><分程序>{;<过程说明部分>;}

	单个过程: procedure p; // 过程首部 var x; // 分程序（变量说明+语句） begin x := 100; end; // 过程说明部分结束 多个过程: procedure p; // 第一个过程 var x; // 分程序（变量说明+语句） begin x := 1; end; procedure q; // 第二个过程 var y; // 分程序（变量说明+语句） begin y := 2; end; // 过程说明部分结束
<过程首部>	::=procedure<标识符>; procedure p;
<语句>	::=<赋值语句> <条件语句> <当型循环语句> <过程调用语句> <读语句> <写语句> <复合语句> <空语句>
<赋值语句>	::=<标识符>:=<表达式> x := 10; sum := a + b * c;
<复合语句>	::=begin<语句>{;<语句>}end begin x := 1; y := 2; z := x + y; end
<空语句>	::=ε
<条件>	::=<表达式><关系运算符><表达式> odd<表达式> （1）表达式 关系符 表达式 a > b x = 0 y # 5 z <= 100 （2）odd 表达式（判断是否奇数） odd(x) odd(a + b)
<表达式>	::=[+ -]<项>{<加减运算符><项>} a + b 100 - x -y + 5 * z
<项>	::=<因子>{<乘除运算符><因子>} a * b x / 2 y * z / 5
<因子>	::=<标识符> <无符号整数> '(<表达式>)',

	(1) 变量名 x (2) 数字 123 (3) (表达式) (a + b) * c 中的 (a + b)
<加减运算符>	::=+ -
<乘除运算符>	::=*/
<关系运算符>	::== < > <= >=
<条件语句>	::=if<条件>then<语句> if x > 10 then x := x + 1;
<过程调用语句>	::=call<标识符> call p;
<当型循环语句>	::=while<条件>do<语句> while odd(b) do call p;
<读语句>	::=read (<标识符>{,<标识符>}); read(x,y);
<写语句>	::=write (<表达式>{,<表达式>}); write(a+b,10);

3 Flex 和 Bison 的使用

3.1 任务描述

学习自动构造词法分析工具 Flex (Fast Lexical Analyzer Generator) 和自动构造语法分析/语义计算工具 Bison (GNU Bison) 的安装、配置及使用方法, 加深对词法分析、语法分析及语义计算工作过程的理解。

3.1.1 Flex

Flex 是自动生成词法分析器的工具, 其工作流程:

正规式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ 最小化 DFA $\rightarrow$ 生成 C 语言词法分析器

- (1) 读入源程序字符流;
- (2) 逐个字符扫描, 尽可能匹配最长的合法单词;
- (3) 根据正则规则切分成 Token (词法单元), 跳过空白、注释, 将 Token 送给 Bison

3.1.2 Bison

Bison 是自动生成语法分析器的工具, 其工作流程:

语言文法 $\rightarrow$ Bison 生成分析表 (LALR (1)) $\rightarrow$ 生成 C 语言语法分析器

- (1) 调用 Flex 的 `yylex()` 函数获得 Token;
- (2) 将 Token 压入分析栈;
- (3) 根据文法规则执行移进/归约;
- (4) 构建语法结构 (表达式、语句、程序);
- (5) 执行语义动作。

Flex+Bison 是用 C 语言实现的、能分析 PL/0 语言的分析器。

Flex 生成一个函数: `yylex()`; Bison 生成一个函数: `yyparse()`;

`yyparse()` $\rightarrow$ 调用 `yylex()` $\rightarrow$ 得到一个 Token $\rightarrow$ 继续语法分析

3.2 实验

完成以下编程任务

任务 1-1: 密文字符频率分析

编写 Flex 描述文件或程序, 实现对给定的密码密文进行字符频率统计 (凯撒密码字符频率统计)。程序应能够读取一行或多行密文输入, 只统计大写字母, 如果遇到小写字母则转换为大写字母进行统计, 并忽略空格、标点符号及数字。

任务 1-2: 编写一个 Flex 描述文件, 识别出指定文本串里的单词、数字和符号 (空格不作处理)。

任务 1-3: 编写一个 Bison 描述文件, 实现具有加法和乘法功能的计算器。

3.3 测试说明

任务 1-1:

输入: KHOOR ZRUOG

输出: K:10%, H:10%, O:30%, R:20%, Z:10%, U:10%, G:10%

任务 1-2:

输入:

```
using namespace std;  
int main()  
{  
    int year = 2023;  
    cout << "hello" << endl;  
    return 0;  
}
```

输出:

using 单词

namespace 单词

std 单词

; 符号

int 单词

main 单词

(符号

) 符号

{ 符号

int 单词

year 单词

= 符号

2023 数字

; 符号

cout 单词

< 符号

< 符号

" 符号

hello 单词

" 符号

< 符号

< 符号

endl 单词

; 符号

return 单词

0 数字

; } 符号

任务 1-3:

输入: $5*7+2$

输出: 37

输入: $9+3*6$

输出: 27

试试在网上找“简单计算器”的 flex 和 bison 联合编程案例。

4 词法分析程序的设计与实现

4.1 任务描述

加深对词法分析工作过程的理解；强化对词法分析方法的掌握；使用 **C/C++语言、Java 语言、Python 语言（或其他编程语言）** 编写 PL/0 编译程序的词法分析程序。并使用自己编写的程序对 PL/0 编程语言程序段进行词法分析。

4.2 实验

（一）必做任务

- （1）能够正确识别关键字、标识符、数字、运算符及界符；
- （2）能自动识别并忽略多行注释/* */ 及 单行注释 //格式的注释信息；
- （3）词法出错处理模块，能够识别出识别非法字符、非法单词等；
- （4）实现正规式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ DFA 最小化的编程实现；
- （5）输出词法单元及其类别；
- （6）词法分析过程中遇到错误后能继续往下识别，并输出错误信息。
- （7）能够将词法分析过程可视化，设计单词分类表，绘制状态转换图，设计词法识别流程图，输出分析结果；

注意：出错处理模块要能够识别以下基本错误。

- （1）识别非法字符：如 @ 、 & 和 ! 等；
- （2）识别非法单词：数字开头的数字字母组合；
- （3）标识符和无符号整数的长度不超过 8 位；

（4）词法分析与语法分析之间的接口设计方法，需要与团队进行协商设计。需要将词法分析的结果存入文件，为语法分析提供输入。或将词法分析的结果保存于设计的数据结构中。

（二）加分项

发现潜在错误或改进空间，并提出优化建议。

4.3 测试说明

输入 1: (正确的词法的测试用例)

```
const a = 10;
var    b, c;
procedure fun1;
    if a <= 10 then
        begin
            c := b + a;
        end;
begin
    read(b);
    while b # 0 do
        begin
            call fun1;
            write(2 * c);
            read(b);
        end
    end
end.
```

输出 1:

(保留字,const)
(标识符,a)
(运算符,=)
(无符号整数,10)
(界符,;)
(保留字,var)
(标识符,b)
(界符,,)
(标识符,c)
(界符,;)
(保留字,procedure)
(标识符,fun1)
(界符,;)
(保留字,if)

(标识符,a)
(运算符,<=)
(无符号整数,10)
(保留字,then)
(保留字,begin)
(标识符,c)
(运算符,:=)
(标识符,b)
(运算符,+)
(标识符,a)
(界符,;)
(保留字,end)
(界符,;)
(保留字,begin)
(保留字,read)
(界符,())
(标识符,b)
(界符,))
(界符,;)
(保留字,while)
(标识符,b)
(运算符,#)
(无符号整数,0)
(保留字,do)
(保留字,begin)
(保留字,call)
(标识符,fun1)
(界符,;)
(保留字,write)
(界符,())
(无符号整数,2)
(运算符,*)
(标识符,c)
(界符,))

(界符,,)
(保留字,read)
(界符,())
(标识符,b)
(界符,,)
(界符,,)
(保留字,end)
(保留字,end)
(界符,,)

输入 2: (含有错误单词、注释的测试用例)

```
const 2a = 123456789;  
  
var    b, c;  
  
//单行注释  
  
/*  
* 多行注释  
*/  
  
procedure function1;  
    if 2a <= 10 then  
        begin  
            c := b + a;  
        end;  
begin  
    read(b);  
    while b @ 0 do  
        begin  
            call function1;  
            write(2 * c);  
            read(b);
```

```
end  
end.
```

输出 2:

```
(保留字,const)  
(非法字符(串),2a,行号:1)  
(运算符,=)  
(无符号整数越界,123456789,行号:1)  
(界符,,)  
(保留字,var)  
(标识符,b)  
(界符,,)  
(标识符,c)  
(界符,,)  
(保留字,procedure)  
(标识符长度超长,function1,行号:10)  
(界符,,)  
(保留字,if)  
(非法字符(串),2a,行号:11)  
(运算符,<=)  
(无符号整数,10)  
(保留字,then)  
(保留字,begin)  
(标识符,c)  
(运算符,:=)  
(标识符,b)  
(运算符,+)  
(标识符,a)  
(界符,,)
```

(保留字,end)

(界符,;)

(保留字,begin)

(保留字,read)

(界符,())

(标识符,b)

(界符,))

(界符,;)

(保留字,while)

(标识符,b)

(非法字符(串),@,行号:17)

(无符号整数,0)

(保留字,do)

(保留字,begin)

(保留字,call)

(标识符长度超长,function1,行号:19)

(界符,;)

(保留字,write)

(界符,())

(无符号整数,2)

(运算符,*)

(标识符,c)

(界符,))

(界符,;)

(保留字,read)

(界符,())

(标识符,b)

(界符,))

(界符,;)

(保留字,end)

(保留字,end)

(界符,.)

注意：正确的测试用例不限于上述例子，但应该包括所有的单词类别；错误的测试用例不限于上述例子，但应该包括所有的错误类型。

5 语法分析程序设计与实现

5.1 任务描述

加深对语法分析工作过程的理解；强化对语法分析方法的掌握；基于词法分析程序，能够采用使用 C/C++语言、Java 语言、Python 语言（或其他编程语言）编写 PL/0 编译程序的词法分析程序。实现 PL/0 编译程序的语法分析程序，并使用自己编写的语法分析程序对 PL/0 语言程序段进行语法分析。

5.2 实验

一、自顶向下的语法分析程序

（一）必做任务

1. 实现自顶向下语法分析程序（递归下降法或表驱动的 LL（1）分析法）；
2. 能够解析 PL/0 程序段，包括：
常量、变量、过程声明、表达式语句、条件语句、循环语句、复合语句；
3. 实现 SELECT 集的编程实现，LL(1) 分析表的编程实现；
4. 实现递归下降法或表驱动的 LL（1）分析法的程序实现，对于表驱动的 LL（1）分析法给出可视化的分析栈分析步骤；
5. 输出语法分析结果，标明语法是否正确；
6. 能检测常见语法错误（如缺少分号、括号不匹配等），并提示错误行号。

（二）加分项

1. 生成语法分析树的可视化；
2. 对输入文法进行左公因子或左递归消除的编程实现；
3. 在检测语法错误的同时，提供修复建议或改进策略。

二、自底向上的语法分析（LR）程序

（一）必做任务

1. 实现自底向上语法分析程序，可选择：LR(0)、SLR(1)、LR(1)和 LALR(1)方法；
2. 能够解析 PL/0 程序段，包括：
常量、变量、过程声明、表达式语句、条件语句、循环语句、复合语句；
3. 实现 LR 类项目集及识别活前缀的状态转换图的自动化生成；
4. 实现 LR 分析表自动化构造；
5. 输出格式应显示：归约/移进操作序列、当前状态和输入符号，语法分析结果（正确/错误）

4. 能检测常见语法错误（如缺少分号、括号不匹配等），并提示错误行号。

（二）加分项

1. 能生成完整的语法分析树并可视化。

5.3 测试说明

输入 1：正确的语法的测试用例

```
const a = 10;

var   b, c;

procedure p;
    if a <= 10 then
        begin
            c := b + a;
        end;
begin
    read(b);
    while b # 0 do
        begin
            call p;
            write(2 * c);
            read(b);
        end
    end.
end.
```

输出：

语法正确

错误语法的测试用例

输入：

```
const a := 10;
```



```
var    b, c;

procedure p;

    if a <= 10 then

        begin

            c := b + a;

        end;

begin

    read(b);

    while b # 0 do

        begin

            call p;

            write(2 * c);

            read(b);

        end

    end.

end.
```

输出:

(语法错误,行号:1)

输入:

```
while b # 0 do

    begin

        call p;

        write(2 * c);

        read(b);

    end

end.
```

输出:

(语法错误,行号:3)

输入:

```
const a = 10;
```

```
var    b, c;

//单行注释

/*
 * 多行注释
 */

procedure p;
    if a <= 10 then
        begin
            c := b + a;
        end;
begin
    read(b);
    while b # 0
        begin
            call p;
            write(2 * c);
            read(b);
        end
end.
```

输出:

(语法错误,行号:17)

输入:

```
const a := 10;
var    b, c d;
```

```
//单行注释
```

```
/*  
* 多行注释  
*/  
  
procedure procedure fun1;  
    if a <= 10  
        begin  
            c = b + a  
        end;  
begin  
    read(b;  
    while b # 0  
        begin  
            call fun1;  
            write 2 * c);  
            read(b);  
        end  
end.  
end.
```

输出:

(语法错误,行号:1)

(语法错误,行号:2)

(语法错误,行号:10)

(语法错误,行号:11)

(语法错误,行号:13)

(语法错误,行号:16)

(语法错误,行号:17)

(语法错误,行号:20)

注意: 测试用例不限于上述例子。

6 语义分析

6.1 任务描述

加深对语义分析工作过程的理解；强化对语义分析方法的掌握；基于词法分析和语法分析程序，能够采 C/C++ 语言、Java 语言、Python 语言（或其他编程语言）实现 PL/0 编译程序的语义分析程序，并使用自己编写的程序对 PL/0 编程语言程序段进行语义分析；能够生成 PL/0 编程语言四元式形式的中间代码。

6.2 常见的四元式格式

编号	四元式代码	含义
(1)	(syss,_,_,_)	程序开始（system start）
(2)	(syse,_,_,_)	程序结束（system end）
(3)	(const,ident,_,_)	声明标识符 ident（常量）
(4)	(var,ident,_,_)	声明标识符 ident（变量）
(5)	(procedure,ident,_,_)	声明标识符 ident（函数）
(6)	(+,A,B,T)	加法运算，将 A+B 的结果赋给 T
(7)	(-,A,B,T)	减法运算，将 A-B 的结果赋给 T
(8)	(*,A,B,T)	乘法运算，将 A*B 的结果赋给 T
(9)	(/,A,B,T)	除法运算，将 A/B 的结果赋给 T
(10)	(<,A,B,T)	小于，将 A<B 的结果赋给 T
(11)	(<=,A,B,T)	小于或等于，将 A<=B 的结果赋给 T
(12)	(>,A,B,T)	大于，将 A>B 的结果赋给 T
(13)	(>=,A,B,T)	大于或等于，将 A>=B 的结果赋给 T
(14)	(#,A,B,T)	不等于，将 A#B 的结果赋给 T
(15)	(=,A,B,T)	等于，将 A=B 的结果赋给 T
(16)	(=,A,_,T)	将 A（的值）赋给常量 T
(17)	(:=,A,_,T)	将 A（的值）赋给变量 T
(18)	(jrop,A,B,P)	当 A rop B 为真时跳转到四元式 P
(19)	(read,A,_,_)	为变量 A 输入值

(20)	(write,A,_,_)	输出 A 的值
(21)	(call,fun,_,A)	调用 fun 函数，返回值存入变量 A
(22)	(call,fun,_,_)	调用 fun 函数，不保存返回值
(23)	(ret,A,_,_)	返回返回值 A
(24)	(ret,_,_,_)	返回，无返回值

6.3 实验

任务 1-1: 基于词法分析程序和自顶向下的语法分析程序，基于 L-翻译模式，编写 PL/0 编译程序的语义分析程序，并生成四元式形式的中间代码。

任务 1-2: 基于词法分析程序和自底向上的语法分析程序，基于 S-翻译模式，编写 PL/0 编译程序的语义分析程序，并生成四元式形式的中间代码。

（一）必做任务

- 1. 能够分析 PL/0 程序段,处理变量声明、赋值语句、条件语句、循环语句和过程调用；
- 2. 能生成基本的控制流跳转和临时变量管理；
- 3. 能正确处理语义错误（如未声明变量、重复声明、类型不匹配）。

（二）加分项

- 1. 能够将四元式执行过程可视化，包括跳转控制流和临时变量使用情况；
- 2. 借助 AI 可辅助生成分析过程图，但能解释四元式生成逻辑。

6.4 测试说明

任务 1-1/1-2:

输入:

```
const a = 10;

var    b, c;

//单行注释

/*
 * 多行注释
*/
```

```
procedure p;  
  if a <= 10 then  
    begin  
      c := b + a;  
    end;  
begin  
  read(b);  
  while b # 0 do  
    begin  
      call p;  
      write(2 * c);  
      read(b);  
    end  
end.  
end.
```

输出:

语义正确

中间代码:

- (1)(syss,_,_,_)
- (2)(const,a,_,_)
- (3)(=,10,_,a)
- (4)(var,b,_,_)
- (5)(var,c,_,_)
- (6)(procedure,p,_,_)
- (7)(j<=,a,10,\$8)
- (8)(+,b,a,c)
- (9)(ret,_,_,_)
- (10)(read,b,_,_)
- (11)(j#,b,0,\$13)

(12)(j:=b,0,\$17)

(13)(call,p,_,_)

(14)(*,2,c,T1)

(15)(write,T1,_,_)

(16)(read,b,_,_)

(17)(syse,_,_,_)

符号表:

const a 10

var b 0

var c 0

procedure p

输入:

```
const a = 10;
```

```
var a, b, c;
```

```
procedure p;
```

```
    if a <= 10 then
```

```
        begin
```

```
            c := b + a;
```

```
        end;
```

```
begin
```

```
    read(b);
```

```
    while b # 0 do
```

```
        begin
```

```
            call p;
```

```
            write(2 * c);
```

```
            read(b);
```

```
        end
```

```
end.
```


输出:

(语义错误,行号:2)

输入:

```
const a = 10;

var b, c;

//单行注释

/*
 * 多行注释
 */

procedure p;
    if a <= 10 then
        begin
            c := b + a;
        end;
begin
    read(p);
    while b # 0 do
        begin
            call p;
            write(2 * c);
            read(b);
        end
    end.

end.
```

输出:

(语义错误,行号:16)

输入:

```
const a = 10;

var   b, c;

//单行注释

/*
 * 多行注释
 */

procedure p;
    if a <= 10 then
        begin
            c := b + a;
        end;
begin
    read(p);
    while b # 0 do
        begin
            call q;
            write(2 * c);
            read(b);
        end
    end.
end.
```

输出:

(语义错误,行号:19)

输入:

```
const a = 10;

var   a, b, c;
```

```
//单行注释
```

```
/*
```

```
* 多行注释
```

```
*/
```

```
procedure p;
```

```
    if a <= 10 then
```

```
        begin
```

```
            c := b + a;
```

```
        end;
```

```
begin
```

```
    read(p);
```

```
    while b # 0 do
```

```
        begin
```

```
            call q;
```

```
            write(2 * d);
```

```
            read(b);
```

```
        end
```

```
end.
```

输出:

(语义错误,行号:2)

(语义错误,行号:16)

(语义错误,行号:19)

(语义错误,行号:20)

注意: 测试用例不限于上述例子。

7 课程设计的组织形式要求

编译原理课程设计以分组的形式通过分工合作来完成。分组的形式既能锻炼学生的系统整体设计能力，又能提升他们相互间的合作组织能力，组队的规模为每组 3-4 人。

8 课程设计完成形式

(1) 编译原理课程设计报告（纸质+电子版）

主要内容有课程设计任务及要求，词法分析程序设计及实现（包括文法、设计原理、数据结构、设计流程图、主要算法、NFA、DFA、分析过程等），语法分析程序设计及实现（包括文法、设计原理、数据结构、设计流程图、主要算法（LR 分析方法要包括识别活前缀的 DFA 等）、分析表、分析过程等），语义分析程序设计及实现（包括文法、设计原理、数据结构、设计流程图、主要算法（L/S 翻译模式）、分析过程等），系统测试、结果及分析，总结。

(2) 验收和答辩

以分组为单位，按照小组分工进行现场演示、答辩、验收。

9 验收及考核

(1) 考核方式及成绩评定标准

总评成绩=报告成绩×40%+考核成绩×60%。

(2) 验收及报告评分参考

评价项目及内涵		
课程目标	评价项目（占比）	评价内涵（优秀标准）
1	作品中的系统设计与应用（30）	根据任务要求，对词法、语法和语义及中间代码生成各个模块功能满足要求，运行正常，完成指定的输入分析，设计有创新，对所实现的系统理解透彻。 评价依据：软件验收

	报告中的系统设计与应用 (20)	利用所学知识对实际工程问题进行分析归纳，设计方案论证充分合理，逻辑清晰，数据结构及算法设计正确。设计报告文字通顺，需求分析、系统设计等描述清晰准确，图表规范，内容充实。 评价依据：课程设计报告
2	作品中的总结与综合能力 (18)	验收准备充分，思路清晰，语言表达准确，论点正确。回答问题基本概念清楚，回答准确。对实验数据及结果分析正确，逻辑清晰。 评价依据：答辩表现
	报告中的总结与综合能力 (20)	对产生的相应结果数据进行了充分分析，给出各种结果的归纳总结。 评价依据：课程设计报告
3	作品中的团队协作 (12)	进度计划、人员分工合理可行，工作态度认真，团队负责人能领导团队按时完成工作。 评价依据：团队成员贡献、项目完成情况、平时工作表现